

Set 17: Python Strings

Skill 17.1: Apply *index* to select a character from a string

Skill 17.2: Apply *slice* to a string

Skill 17.3: Interpret what is meant by *mutability*

Skill 17.4: Apply escape characters

Skill 17.5: Iterate over strings

Skill 17.6: Apply the *in* keyword

Skill 17.1: Apply index to select a character from a string

Skill 17.1 Concepts

In Python, the way we store something like a word, a sentence, or even a whole paragraph is as a string. A string is a sequence of characters contained within a pair of 'single quotes' or "double quotes". A string can be any length and can contain any letters, numbers, symbols, and spaces.

A string can be thought of as a list and like any other list, each character in a string has an index. We can select specific letters from this string using the *index*.

Code	Output
<pre>favorite_fruit = "blueberry" print(favorite_fruit[0]) print(favorite_fruit[1]) print(favorite_fruit[2]) print(favorite_fruit[3]) print(favorite_fruit[4])</pre>	<pre>b l u e b</pre>

Notice, in the above example strings are “zero” indexed just like lists. It’s also important to note that indices of strings must be integers. If we were to try to select a non-integer index we would get a `TypeError`,

Code	Output
<pre>favorite_fruit = "blueberry" print(favorite_fruit[1.5])</pre>	<pre>TypeError: string indices must be integers, not 'float'</pre>

Skill 17.1 Exercise 1

Skill 17.2: Apply slice to a string

Skill 17.2 Concepts

Not only can we select a single character from a string, but we can also select entire chunks of characters from a string. We can do this with the following syntax,

```
string[first_index:last_index:step]
```

first_index: The starting index of the substring. The character at this index is included in the substring. If start is not included, it is assumed to equal to 0.

last_index: The terminating index of the substring. The character at this index is not included in the substring. If end is not included, or if the specified value exceeds the string length, it is assumed to be equal to the length of the string by default.

step: Every "step" character after the current character to be included. The default value is 1. If step is not included, it is assumed to be equal to 1.

This is called *slicing* a string. When we slice a string we are creating a substring - a brand new string that starts at (and includes) the first_index and ends at (but excludes) the last_index.

Below are some examples of how to access parts of the string, "Sparky the dog".

Position	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Letter	S	p	a	r	k	y		t	h	e		d	o	g

Code	Output	Explanation
<pre>pet = "Sparky the dog" letters = pet[0:1] print(letters)</pre>	S	The character at index 0 is printed, the character at index 1 is not printed
<pre>pet = "Sparky the dog" letters = pet[0:2] print(letters)</pre>	Sp	All the characters starting at index 0 and up to index 2 are printed
<pre>pet = "Sparky the dog" letters = pet[3] print(letters)</pre>	r	Only the character at index 3 is printed
<pre>pet = "Sparky the dog" letters = pet[3:] print(letters)</pre>	rky the dog	All the characters starting at index 3 through the end of the string are printed
<pre>pet = "Sparky the dog" letters = pet[:3] print(letters)</pre>	Spa	The first 3 letters are printed
<pre>pet = "Sparky the dog" letters = pet[::2] print(letters)</pre>	Sak h o	Every other letter is printed
<pre>pet = "Sparky the dog" letters = pet[-1] print(letters)</pre>	g	The last letter is printed
<pre>pet = "Sparky the dog" letters = pet[:-1] print(letters)</pre>	Sparky the do	All the letters, except for the last letter is printed

Skill 17.2 Exercises 1 & 2

Skill 17.3: Interpret what is meant by an *mutability*

Skill 17.3 Concepts

So far in this lesson, we've been selecting characters from strings, slicing strings, and concatenating strings. Each time we perform one of these operations we are creating an entirely new string. This is because strings are *immutable*. This means that we cannot change a string once it is created. We can use it to create other strings, but we cannot change the string itself.

This property, generally, is known as *mutability*. Data types that are *mutable* can be changed, and data types, like strings, that are *immutable* cannot be changed.

[Skill 17.3 Exercise 1](#)

Skill 17.4: Apply escape characters to strings

Skill 17.4 Concepts

Recall that escape sequences are used to signal an alternative interpretation of a series of characters. In JavaScript, a character preceded by a *backslash* (`\`) is an escape sequence.

Below are some commonly used escape sequences in JavaScript

`\t`: Inserts a tab

This sequence inserts a *tab* in the text where it's used.

`\n`: Inserts a new line

This sequence inserts a *new line* in the text where it's used.

`\'`: Inserts a single quote

This sequence inserts a *single quote* character in the text where it's used.

`\"`: Inserts a double quote

This sequence inserts a *double quote* character in the text where it's used.

`\\`: Inserts a backslash

This sequence inserts a *backslash* character in the text where it's used.

Below are a few examples of how escape sequences can be implemented,

Code	Output
<pre>msg = "Here is one line \n and here is another" print(msg)</pre>	Here is one line and here is another
<pre>print("What is the \"right\" way?")</pre>	What is the "right" way?
<pre>msg2 = "Everything\t\tis\t\t\ttabbed" print(msg2)</pre>	Everything is tabbed

Skill 17.4 Exercise 1

Skill 17.5: Iterate over strings

Skill 17.5 Concepts

Because strings are lists, that means we can iterate over a string using the same loops we learned about before. Let's look at an example that prints out each letter in a word.

Code	Output
<pre>def print_each_letter(word): for letter in word: print(letter) favorite_color = "blue" print_each_letter(favorite_color)</pre>	b l u e

Now that we know how to iterate through strings, we can explore the potential of strings by including conditional statements inside the iterations.

Take a look at the following code,

Code	Output
<pre>favorite_fruit = "blueberry" counter = 0 for character in favorite_fruit: if character == "b": counter = counter + 1 print(counter)</pre>	2

This code above counts the number of b's in the string "blueberry". Let's take a moment and break down what exactly this code is doing.

First, we define our string, *favorite_fruit*, and a variable called *counter*, which we set equal to zero. Then the for loop iterates through each character in *favorite_fruit* and compares it to the letter b.

Next, each time a character equals *b* the code will increase the variable counter by one.

Finally, once all characters have been checked, the code prints the counter, telling us how many b's are in "blueberry".

[Skill 17.5 Exercises 1 & 2](#)

Skill 17.6: Apply the *in* keyword

Skill 17.6 Concepts

There's an easier way to iterate over a string and check if a letter is present.

We can do this type of check more efficiently using the *in* keyword. *in* checks if one string is part of another string. Here is what the syntax of *in* looks like

```
letter in string
```

Here, *letter in string* is a Boolean expression that is True if the string letter is in the string word. Here are some examples,

Code	Output
<pre>print("e" in "blueberry")</pre>	True
<pre>print("a" in "blueberry")</pre>	False
<pre>print("blue" in "blueberry")</pre>	True
<pre>print("blue" in "strawberry")</pre>	False

It can be helpful to include more than one Boolean expression in the same line of code. To do this, use *and* or *and not* in between the Boolean expressions.

Code	Output
<pre>print("e" in "blueberry" and "e" in "carrot")</pre>	False
<pre>print("e" in "blueberry" and not "e" in "carrot")</pre>	True

The first example above is False because ONE of the expressions was False; there is no "e" in "carrot". The second example is True because there is an "e" in "blueberry" and not an "e" in "carrot"; both expressions are True.

[Skill 17.6 Exercises 1 and 2](#)